NLP Assignment 5

Fine-Tuning DistilBERT for POS Tagging

Introduction

In this assignment, I worked on a Natural Language Processing task called token classification.

The goal was to build a model that can understand each word in a sentence and assign a grammatical label such as noun, verb, adjective, etc.

To achieve this, I used a transformer-based model called DistilBERT. This task helped me understand how modern NLP models process text and perform sequence labeling.

```
!pip install datasets==2.16.1 transformers seqeval conllu
```

```
Requirement already satisfied: datasets==2.16.1 in /usr/local/lib/python3.12/dist-packages (2.16.1)
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: seqeval in /usr/local/lib/python3.12/dist-packages (1.2.2)
Requirement already satisfied: conllu in /usr/local/lib/python3.12/dist-packages (6.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (3.25.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (2.0.2)
Requirement already satisfied: pyarrow>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (18.1.0)
Requirement already satisfied: pyarrow-hotfix in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (0.7)
Requirement already satisfied: dill<0.3.8,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (0.3.7)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (2.2.2)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (2.32.4)
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (3.6.0)
Requirement already satisfied: multiprocess in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (0.70.15)
Requirement already satisfied: fsspec<=2023.10.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2023.
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (3.13.4)
Requirement already satisfied: huggingface-hub>=0.19.4 in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (1.8.0
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets==2.16.1) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: scikit-learn>=0.21.3 in /usr/local/lib/python3.12/dist-packages (from seqeval) (1.6.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.16.
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.16.1) (1.4
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.16.1) (26.1.0
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.16.1) (1.
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.16.1) (
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.16.1) (0.4
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.16.1) (1.
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.19.4->da
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.19.4->datase
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.19.4->datasets==2.16.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.19.4
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->datas
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->datasets==2.16.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->datasets==2
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->datasets==2
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3->seqeval) (1.1
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3->seqeval) (1.
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3->seqev
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==2.16.1)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==2.16.1) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==2.16.1) (2025.3
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.19.4-
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggi
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->dataset
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.19.4->dat
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.19.
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.19.4->dat
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.1
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggi
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->hug
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->
```

```
from datasets import load_dataset
```

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForTokenClassification, TrainingArguments, Trainer
import numpy as np
from seqeval.metrics import classification_report
from transformers import DataCollatorForTokenClassification
```

Task 1: Dataset Selection

For this assignment, I selected the **Universal Dependencies (en_ewt)** dataset.

This dataset is widely used for POS tagging tasks and contains annotated sentences with grammatical labels.

Label Categories:

Some common POS tags include:

- NOUN
- VERB
- ADJ
- ADV
- PRON
- DET

```
dataset = load_dataset("universal_dependencies", "en_ewt", trust_remote_code=True)
print(dataset)
```

```
DatasetDict({
    train: Dataset({
        features: ['idx', 'text', 'tokens', 'lemmas', 'upos', 'xpos', 'feats', 'head', 'deprel', 'deps', 'misc'],
        num_rows: 12543
    })
    validation: Dataset({
        features: ['idx', 'text', 'tokens', 'lemmas', 'upos', 'xpos', 'feats', 'head', 'deprel', 'deps', 'misc'],
        num_rows: 2002
    })
    test: Dataset({
        features: ['idx', 'text', 'tokens', 'lemmas', 'upos', 'xpos', 'feats', 'head', 'deprel', 'deps', 'misc'],
        num_rows: 2077
    })
})
```

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

Task 2: Data Preprocessing

In this step, I converted text into tokens using the BERT tokenizer.

One important challenge here is aligning labels with tokens because a single word can be split into multiple subwords.

To handle this:

- Special tokens are assigned -100
- Subword tokens are ignored during training

```
label_list = dataset["train"].features["upos"].feature.names

def tokenize_and_align_labels(example):
    tokenized_input = tokenizer(example["tokens"], truncation=True, is_split_into_words=True)

    labels = []
    word_ids = tokenized_input.word_ids()
    previous_word_idx = None

    for word_idx in word_ids:
        if word_idx is None:
            labels.append(-100)
        elif word_idx != previous_word_idx:
            labels.append(example["upos"][word_idx])
        else:
            labels.append(-100)

        previous_word_idx = word_idx
```

```
    tokenized_input["labels"] = labels
    return tokenized_input

tokenized_dataset = dataset.map(tokenize_and_align_labels)
```

```
Map: 100%                                    2002/2002 [00:02<00:00, 955.23 examples/s]
```

```
print(tokenized_dataset["train"][0].keys())
print("Input IDs:", tokenized_dataset["train"][0]["input_ids"][:10])
print("Attention Mask:", tokenized_dataset["train"][0]["attention_mask"][:10])
print("Labels:", tokenized_dataset["train"][0]["labels"][:10])
```

```
dict_keys(['idx', 'text', 'tokens', 'lemmas', 'upos', 'xpos', 'feats', 'head', 'deprel', 'deps', 'misc', 'input_ids', 'token_typ
Input IDs: [101, 2632, 1011, 23564, 2386, 1024, 2137, 2749, 2730, 21146]
Attention Mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Labels: [-100, 10, 1, 10, -100, 1, 6, 0, 16, 10]
```

Optimizing Training Time

To make training faster, a smaller subset of the dataset is used.

```
small_train = tokenized_dataset["train"].select(range(1000))
small_val = tokenized_dataset["validation"].select(range(200))
```

Task 3: Model Setup

Here, I used DistilBERT with a token classification head.

The number of labels is set based on the dataset.

```
id2label = {i: label for i, label in enumerate(label_list)}
label2id = {label: i for i, label in enumerate(label_list)}

model = AutoModelForTokenClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=len(label_list),
    id2label=id2label,
    label2id=label2id
)
```

```
Loading weights: 100%                    100/100 [00:00<00:00, 433.13it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForTokenClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_transform.weight   | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

Task 4: Training

The model is trained using the Hugging Face Trainer API.

I used a small number of epochs to keep training time reasonable.

```
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    num_train_epochs=1,
    weight_decay=0.01,
```

```
    logging_steps=50,
    save_strategy="no"
)

data_collator = DataCollatorForTokenClassification(tokenizer)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=small_train,
    eval_dataset=small_val,
    data_collator=data_collator
)

trainer.train()
```

[63/63 06:52, Epoch 1/1]

Step	Training Loss
50	2.178386

```
TrainOutput(global_step=63, training_loss=2.0645635998438276, metrics={'train_runtime': 419.9311, 'train_samples_per_second': 2.381, 'train_steps_per_second': 0.15, 'total_flos': 16248554292960.0, 'train_loss': 2.0645635998438276, 'epoch': 1.0})
```

Task 5: Evaluation

Model performance is evaluated using precision, recall, and F1-score.

```
predictions, labels, _ = trainer.predict(tokenized_dataset["validation"])

preds = np.argmax(predictions, axis=2)

true_labels = []
true_preds = []

for pred, lab in zip(preds, labels):
    temp_pred = []
    temp_lab = []

    for p, l in zip(pred, lab):
        if l != -100:
            temp_pred.append(label_list[p])
            temp_lab.append(label_list[l])

    true_preds.append(temp_pred)
    true_labels.append(temp_lab)

print(classification_report(true_labels, true_preds))
```

```
              precision    recall  f1-score   support

         ART       1.00      0.13      0.23       630
        CONJ       0.96      0.14      0.24      1228
          DJ       0.72      0.07      0.13      1682
          DP       0.60      0.80      0.69      1974
          DV       0.33      0.03      0.06      1071
         ERB       0.52      0.54      0.53      2693
          ET       0.54      0.82      0.65      1873
         NTJ       0.00      0.00      0.00       112
         OUN       0.42      0.59      0.49      3694
         RON       0.77      0.47      0.58      2151
        ROPN       0.50      0.63      0.56      1344
          UM       0.00      0.00      0.00       350
        UNCT       0.82      0.88      0.85      2951
          UX       0.76      0.41      0.53      1406
          YM       0.00      0.00      0.00        77
           _       0.00      0.00      0.00       404

   micro avg       0.58      0.51      0.55     23640
   macro avg       0.50      0.34      0.35     23640
weighted avg       0.61      0.51      0.50     23640
```

Task 6: Inference

Testing the model on a custom sentence.

```
def predict(sentence):
    tokens = sentence.split()
    inputs = tokenizer(tokens, return_tensors="pt", is_split_into_words=True)

    outputs = model(**inputs).logits
    predictions = np.argmax(outputs.detach().numpy(), axis=2)

    print("\nPrediction:\n")
    for token, pred in zip(tokens, predictions[0]):
        print(f"{token:15} → {label_list[pred]}")

predict("John works at Google in California")
predict("The quick brown fox jumps over the lazy dog")
```

```
Prediction:

John            → PRON
works           → PROPN
at              → NOUN
Google          → ADP
in              → NOUN
California      → ADP

Prediction:

The             → PRON
quick           → DET
brown           → PROPN
fox             → PROPN
jumps           → NOUN
over            → VERB
the             → ADP
lazy            → DET
dog             → NOUN
```

Task 7: Comparison

POS Tagging

- Assigns grammatical roles (noun, verb, etc.)
- Works at word level
- Easier to implement

Chunking

- Groups words into phrases
- Works at phrase level
- Slightly more complex

Final Comparison

POS tagging provides basic understanding, while chunking gives deeper structural meaning.

Task 8: Report

Difference between POS Tagging and Chunking

POS tagging focuses on individual words, while chunking focuses on phrases.

Challenges Faced

- Label alignment with tokens
- Handling subwords
- Understanding transformer workflow

Observations

- Transformers give high accuracy

- Preprocessing plays a very important role

Conclusion

This assignment helped me understand how NLP models process language step by step.

Note

Initially, some datasets caused compatibility issues due to updates in the datasets library.

After resolving dependencies and using proper configurations, the model was successfully trained and evaluated.